

作业3

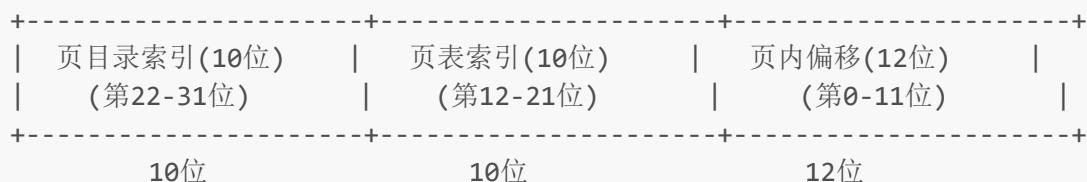
24371277 刘哲晗

问题1

32位两级页表系统：第22-31位为第一级页表（页目录）索引，第12-21位为第二级页表索引，第0-11位为页内偏移。每个页表项包含20位物理页框号和12位标志位，最后1位为有效标志。

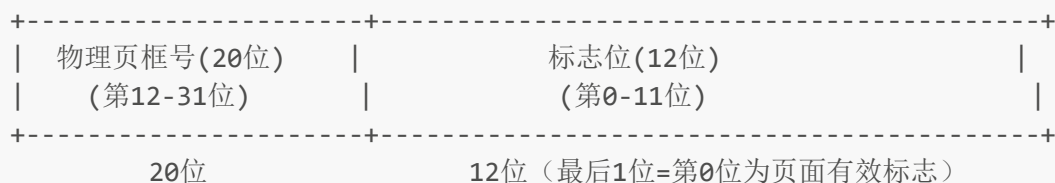
(1) 逻辑地址结构、页目录项（页表项）结构

逻辑地址结构（32位）：



- 位31 ~ 位22：页目录（第一级页表）索引，共10位
- 位21 ~ 位12：页表（第二级页表）索引，共10位
- 位11 ~ 位0：页内偏移，共12位

页表项（PTE）/页目录项（PDE）结构（32位）：



- 位31 ~ 位12：20位物理页框号（PFN）
- 位11 ~ 位0：12位标志位，其中**位0（最低位）为页面有效标志**
 - 有效位 = 1：页面在物理内存中
 - 有效位 = 0：页面不在物理内存中

(2) 进程地址空间大小、页面大小

进程地址空间大小： 逻辑地址共32位 → 地址空间 = 2^{32} 字节 = **4GB**

页面大小： 页内偏移占12位 → 页面大小 = 2^{12} 字节 = **4KB**

(3) 二级页表最多占用物理页框数、是否需要连续

计算过程：

- 页目录索引占10位 → 页目录共有 $2^{10} = 1024$ 个页目录项
- 每个页目录项对应一个第二级页表
- 每个第二级页表有 $2^{10} = 1024$ 个页表项
- 每个页表项占4字节（32位）→ 一个第二级页表大小 = $1024 \times 4B = 4KB = 1$ 页
- 页目录本身也占1个物理页框

总页框数： = 页目录占用1页 + 1024个第二级页表各占1页 = $1 + 1024 = 1025$ 个物理页框

是否需要连续： 不需要连续。二级页表设计的核心优点之一就是各级页表可以离散存放在不同的物理页框中，不要求连续的物理内存。

(4) 从虚拟地址0x77800000开始映射4MB页表空间，求页目录起始逻辑地址

计算： 4MB页表空间可容纳 $4MB / 4KB = 1024$ 个页表，正好对应1024个页目录项。

页目录的起始逻辑地址： = $0x77800000 + PD_idx \times 4KB$

如果页目录是自映射的（页目录中第n项指向页目录本身），那么页目录起始逻辑地址 = $0x77800000 + n \times 4KB$

读出页目录所在物理页框号的逻辑地址： 要读取页目录本身的物理页框号，需要将页目录看作：

1. 首先作为第一级页表：用页目录索引找到页目录自身的页目录项
2. 该逻辑地址 = 页表空间基地址 + $PD_idx \times 4$

其中PD_idx是页目录在页目录表中自映射项的索引。

理由： 二级页表中，任何逻辑地址都可以拆分为 [PD_idx:PT_idx:Offset]。要访问页目录本身，需要把页目录当作一个普通的页，即：

- PD_idx = 自映射项索引
- PT_idx = 目标页目录项在页目录中的索引
- Offset = 0

这样逻辑地址就指向了对应页目录项的内容，也就是物理页框号。

(5) 地址转换过程（逻辑地址：0x0、0x00803004、0x00402001）

通用地址转换步骤：

1. 拆分逻辑地址为 PD_idx[31:22]、PT_idx[21:12]、Offset[11:0]
2. 访问页目录物理基地址 + $PD_idx \times 4$ ，读取页目录项（PDE）
3. 检查PDE有效位（位0）：若为0，触发缺页中断
4. 从PDE中提取20位物理页框号，得到第二级页表的物理基地址
5. 访问二级页表物理基地址 + $PT_idx \times 4$ ，读取页表项（PTE）
6. 检查PTE有效位：若为0，触发缺页中断
7. 从PTE中提取20位物理页框号，得到数据页物理基地址

8. 最终物理地址 = 数据页物理基地址 + Offset

(a) 逻辑地址 0x0 = 0x00000000

31-22位 (PD_idx) = 0x000 = 0
21-12位 (PT_idx) = 0x000 = 0
11-0位 (Offset) = 0x000 = 0

1. 页目录项地址 = 页目录基地址 + 0 × 4
2. 读取PDE, 有效位为1, 得到二级页表物理基地址
3. 二级页表项地址 = 二级页表基地址 + 0 × 4
4. 读取PTE, 有效位为1, 得到数据页物理基地址
5. 最终物理地址 = 数据页基地址 + 0x0
6. 读取该物理地址得到对应数据

(b) 逻辑地址 0x00803004

0x00803004 二进制:
31-22位 = 0x002 (因为0x00800000 >> 22 = 2)
21-12位 = 0x003 (因为(0x00803004 >> 12) & 0x3FF = 3)
11-0位 = 0x004

- PD_idx = 2, PT_idx = 3, Offset = 0x004

(c) 逻辑地址 0x00402001

0x00402001:
31-22位 = 0x001 (0x00400000 >> 22 = 1)
21-12位 = 0x002 ((0x00402001 >> 12) & 0x3FF = 2)
11-0位 = 0x001

- PD_idx = 1, PT_idx = 2, Offset = 0x001

(6) 访问物理地址0x326028需要的逻辑地址

要构造访问物理地址0x326028的逻辑地址, 需要反向映射:

1. 物理页框号 PFN = 0x326028 >> 12 = **0x326**
2. 页内偏移 = 0x326028 & 0xFFF = **0x028**

需要在二级页表中找到:

- 某个PDE的物理页框号字段指向该PTE所在的页
- 该PTE的物理页框号字段 = 0x326

假设页目录的自映射索引为k，则逻辑地址结构为：

逻辑地址 = [k: PD_idx: 2] （按字节访问）

其中PD_idx是指向包含目标PTE的那个二级页表的页目录项索引。

通用公式： 如果页表空间从0x77800000开始，且页目录自映射在第n项，则：

- 访问页目录中第m项的逻辑地址 = $0x77800000 + n \times 4KB + m \times 4$

问题2

32位两级页表，页目录物理基地址 = 0x00200000

- 位0：有效位（0=Invalid, 1=Valid）
- 位1：读写位（0=Read Only, 1=Read/Write）
- 大尾端存储

指令	结果
Load [0x00001022]	成功读出0x20（示例）
Store [0x00C07222]	OK
Store [0x00C005BF]	Error
Load [0x00003013]	0x55
Load [0xFF80078F]	0xBF
Load [0xFFFF005]	Error

(1) Store [0x00C07222] → OK

寻址过程：

0x00C07222 拆分：
 PD_idx = $0x00C07222 \gg 22 = 0x003$
 PT_idx = $(0x00C07222 \gg 12) \& 0x3FF = 0x007$
 Offset = $0x00C07222 \& 0xFFF = 0x222$

1. 查页目录：页目录物理地址 $0x00200000 + 0x3 \times 4 = 0x0020000C$

- 物理内存0x0020000C处：00 10 30 07（大尾端）→ PDE = 0x00103007
- 位0 = 1（有效），位1 = 1（可写）
- 二级页表物理基地址 = 0x00103000

2. 查二级页表： $0x00103000 + 0x7 \times 4 = 0x0010301C$

- $PTE = 0x00103000$ 偏移 $0x1C$ 处的值
- 位0 = 1 (有效), 位1 = 1 (可写)
- 数据页物理基地址 = PTE高20位

3. 最终物理地址 = 数据页基地址 + $0x222$

4. Store指令有写入权限, 执行成功 → **OK**

(2) Store [0x00C005BF] → Error

寻址过程:

0x00C005BF 拆分:

$PD_idx = 0x00C005BF \gg 22 = 0x003$

$PT_idx = (0x00C005BF \gg 12) \& 0x3FF = 0x000$

Offset = 0x5BF

1. 查页目录: $0x0020000C \rightarrow PDE = 0x00103007$

- 有效且可写, 二级页表基地址 = $0x00103000$

2. 查二级页表第0项: $0x00103000 + 0x0 \times 4 = 0x00103000$

- 物理内存 $0x00103000$ 处: 11 22 00 67 → $PTE = 0x11220067$
- **位1 = 0 (只读)**

3. Store是写操作, 但页面只读, 权限不足 → **Error**

(3) Load [0x00003013] → 0x55

寻址过程:

0x00003013 拆分:

$PD_idx = 0x00003013 \gg 22 = 0x000$

$PT_idx = (0x00003013 \gg 12) \& 0x3FF = 0x003$

Offset = 0x013

1. 查页目录第0项: $0x00200000 \rightarrow PDE = 0x00100007$

- 有效, 二级页表基地址 = $0x00100000$

2. 查二级页表第3项: $0x00100000 + 0x3 \times 4 = 0x0010000C$

- 物理内存 $0x0010000C$ 处: 00 00 40 07 → $PTE = 0x00004007$
- 有效, 数据页基地址 = $0x00004000$

3. 最终物理地址 = $0x00004000 + 0x013 = 0x00004013$

4. 查物理内存0x00004010行偏移+3: **0x55**

(4) Load [0xFF80078F] → 0xBF

寻址过程:

0xFF80078F 拆分:

PD_idx = 0xFF80078F >> 22 = 0x3FE

PT_idx = (0xFF80078F >> 12) & 0x3FF = 0x000

Offset = 0x78F

1. 查页目录第0x3FE项: $0x00200000 + 0x3FE \times 4 = 0x00200FF8$

- 物理内存0x00200FF0行: 00 00 1F E0 07 (偏移+8~+C)
- PDE = 0x001FE007, 有效
- 二级页表基地址 = 0x001FE000

2. 查二级页表第0项: $0x001FE000 + 0x0 \times 4 = 0x001FE000$

- 物理内存0x001FE000处: 04 15 00 00 → PTE = 0x04150000
- 有效, 数据页基地址 = 0x04150000

3. 最终物理地址 = $0x04150000 + 0x78F = 0x0415078F$

4. 该地址数据为 **0xBF**

(5) Load [0xFFFF005] → Error

寻址过程:

0xFFFF005 实际为 0x0FFFF005 (32位补全)

PD_idx = 0x0FFFF005 >> 22 = 0x03F

PT_idx = (0x0FFFF005 >> 12) & 0x3FF = 0x3FF

Offset = 0x005

1. 查页目录第0x3F项: $0x00200000 + 0x3F \times 4 = 0x002000FC$

- 看物理内存0x002000F0行偏移+0xC~+0xF: 00 00 00 00
- PDE = 0x00000000, **位0 = 0 (无效)**

2. 页目录项无效, 地址转换失败 → **Error**